

LC 139 & LC 140: Word Break I & II

Authored by: Arpit Raj, LNMIIT Jaipur

Date: August 30, 2026

Asked in: Dropbox, Zillow, Adobe, Park+, Mahindra, Microsoft, Capgemini

Problem Statement

Given a string **s** and a dictionary of strings, need to add partition/space in such a way that each word is a valid dictionary word, need to return all such sentences in order.

Example Walkthrough

Input:

S = aadya iamobsessed

dict → aadya, ai, obsessed, mob, i, am, oody, a, sessed

Output (O/P):

1. oody a i a mob sessed
2. aadya i a mob sessed
3. aadya ai a mob sessed
4. oody a i am obsessed
5. aadya i am obsessed
6. oody ai am obsessed

Dynamic Programming Approach

See this as a **partition DP** problem:

[0 j-1]

already solved

[j i-1]

new word

If $s[j \dots i-1]$ is present in dict **and** $dp[j]$ was true, then $dp[i]$ is true.

To solve LC 139 (Word Break - I) we just need to tell if a string can be segmented or not, so simply return $dp[n]$.

C++ Implementation (LC 139)

```
class Solution {
public:
    bool wordBreak(string s, vector<string>& wordDict) {
        int n = s.size();

        // for O(1) lookup
        unordered_set<string> st;
        for(auto &word : wordDict) {
            st.insert(word);
        }

        vector<bool> dp(n+1, false);
        // dp[i] tells if first i letters can be segmented
        dp[0] = true;

        for(int i = 1; i <= n; i++) {
            for(int j = i-1; j >= 0; j--) {
                if(dp[j] == false) continue;
                // [0.....j-1][j.....i-1];

                string wrd = s.substr(j, i-j);
                if(st.find(wrd) != st.end()) {
                    dp[i] = true;
                    break;
                }
            }
        }

        return dp[n];
    }
};
```